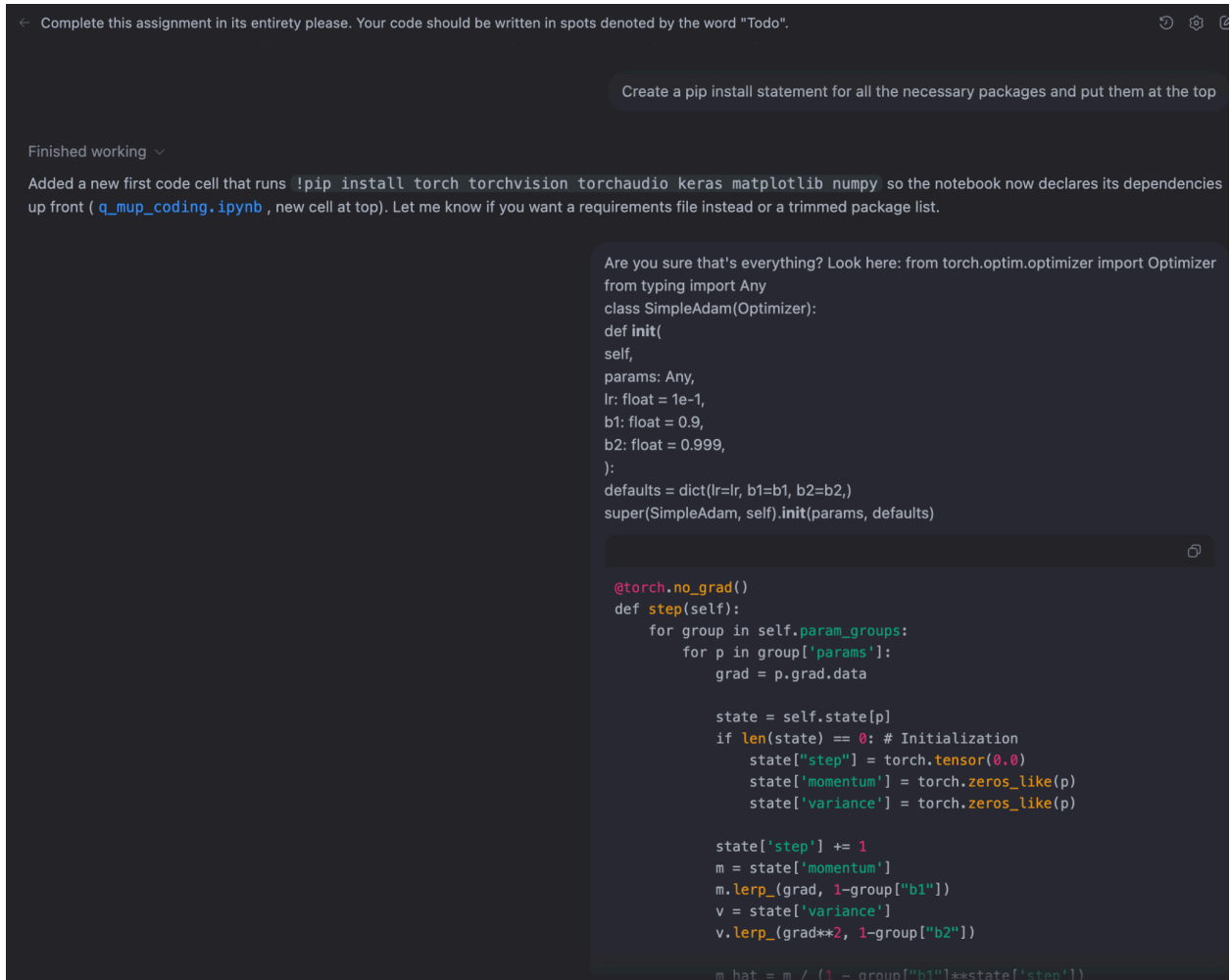


There are two main components in this assignment that I wanted Codex to solve: the coding portions and the answer portions. I first tried asking it to solve and implement everything at once, without any specificity towards examining figures/images.



The screenshot shows a Jupyter Notebook interface. At the top, a message says: "Complete this assignment in its entirety please. Your code should be written in spots denoted by the word 'Todo'." Below this, a code cell is shown with the following content:

```
Create a pip install statement for all the necessary packages and put them at the top
```

Below the code cell, a message says: "Finished working".

Below the message, a text block says: "Added a new first code cell that runs `!pip install torch torchvision torchaudio keras matplotlib numpy` so the notebook now declares its dependencies up front (`q_mup_coding.ipynb` , new cell at top). Let me know if you want a requirements file instead or a trimmed package list."

Below the text block, a code cell is shown with the following content:

```
Are you sure that's everything? Look here: from torch.optim.optimizer import Optimizer
from typing import Any
class SimpleAdam(Optimizer):
    def init(
        self,
        params: Any,
        lr: float = 1e-1,
        b1: float = 0.9,
        b2: float = 0.999,
    ):
        defaults = dict(lr=lr, b1=b1, b2=b2,)
        super(SimpleAdam, self).init(params, defaults)

@torch.no_grad()
def step(self):
    for group in self.param_groups:
        for p in group['params']:
            grad = p.grad.data

            state = self.state[p]
            if len(state) == 0: # Initialization
                state["step"] = torch.tensor(0.0)
                state['momentum'] = torch.zeros_like(p)
                state['variance'] = torch.zeros_like(p)

            state['step'] += 1
            m = state['momentum']
            m.lerp_(grad, 1-group["b1"])
            v = state['variance']
            v.lerp_(grad**2, 1-group["b2"])

            m_hat = m / (1 - group["b1"]**state['step'])
```

The answers and code were filled in and were seemingly correct, but lacked some specificity.

CODEx

← Complete this assignment in its entirety please. Your code should be written in spots denoted by the word "Todo".



Complete this assignment in its entirety please. Your code should be written in spots denoted by the word "Todo".

Finished working ▾

Key Changes

- Added a second invocation of `train_one_step` with wider early layers to recreate the activation-delta plot after changing widths, satisfying part (a)'s plotting request (`q_mup_coding.ipynb (line 194)`).
- Rebuilt `train_one_step_matrices` to capture Frobenius, spectral, and RMS-RMS induced norms (with readable layer labels) so the weight-update analysis in part (b) now works end-to-end (`q_mup_coding.ipynb (line 229)`).
- Implemented the muP-aware optimizer by scaling each parameter's learning rate by its fan-in, with the requested bias/input/output fudge factors, yielding uniform activation deltas across widths (`q_mup_coding.ipynb (line 308)`).
- Added graph-level muP parameterization by normalizing each hidden layer's pre-activations by `sqrt(fan_in)` inside `ScaledMLP`, enabling Adam to reproduce the uniform curves without optimizer changes (`q_mup_coding.ipynb (line 404)`).
- Filled in the simplified Shampoo update by orthogonalizing dense updates via SVD, so you can now contrast Frobenius vs. spectral norms for Adam vs. Shampoo (`q_mup_coding.ipynb (line 556)`).

Notes

- Tests/plots were not rerun here (not requested); please execute the relevant notebook cells to regenerate figures with the new logic.

Can you fill in all the the "Answer:" after the questions as well please

Finished working ▾

Added narrative responses for every "Answer" prompt:

- Documented the network dimensions and behavior of RMS activation deltas before and after width changes (`q_mup_coding.ipynb (line 182)`).
- Explained why the RMS-RMS induced norm best matches activation changes (`q_mup_coding.ipynb (line 219)`).
- Clarified how scaling a layer's output is equivalent to adjusting its learning rate under Adam/SignGD (`q_mup_coding.ipynb (line 387)`).
- Described the learning-rate sweep behavior for plain Adam vs. muP scaling (`q_mup_coding.ipynb (line 436)`).
- Summarized the Frobenius vs. spectral norm relationship for Adam and Shampoo updates (`q_mup_coding.ipynb (line 614)`).

No notebook cells were executed; please re-run the relevant ones in your environment if you want to regenerate plots.

Create a pip install statement for all the necessary packages and put them at the top

I decided to install all the necessary dependencies so that this specificity issue could be fixed.

CODEX

← Complete this assignment in its entirety please. Your code should be written in spots denoted by the word "Todo".

```
def step(group, state):
    m = state['momentum']
    m.lerp(grad, 1-group["b1"])
    v = state['variance']
    v.lerp(grad**2, 1-group["b2"])

    m_hat = m / (1 - group["b1"]**state['step'])
    v_hat = v / (1 - group["b2"]**state['step'])
    u = m_hat / (torch.sqrt(v_hat) + 1e-16)

    p.add_(u, alpha=-group['lr'])
    return None
```

Finished working ▾

The new first cell in `q_mup_coding.ipynb` (`q_mup_coding.ipynb` (line 1)) installs `torch`, so `from torch.optim.optimizer import Optimizer` (and the rest of the `torch` stack) is covered. The other imports in the notebook— `numpy`, `matplotlib`, and `keras.datasets` —are also satisfied by the same `!pip install torch torchvision torchaudio keras matplotlib numpy` line. `typing` is part of the Python standard library, so it needs no extra package. Let me know if you prefer pinning specific versions or trimming that install list.

```
----> 5 from keras.datasets import mnist
6 import matplotlib.pyplot as plt
8 get_ipython().run_line_magic('matplotlib', 'inline')
File ~/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages/keras/init.py:7
1 """DO NOT EDIT.
2
3 This file was autogenerated. Do not edit it by hand,
4 since your modifications would be overwritten.
5 """
----> 7 from keras import _tf_keras as _tf_keras
8 from keras import activations as activations
9 from keras import applications as applications
File ~/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages/keras/_tf_keras/init.py:1
----> 1 from keras._tf_keras import keras
File ~/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages/keras/_tf_keras/keras/init.py:7
1 """DO NOT EDIT.
2
...
----> 13 from tensorflow.python.trackable.data_structures import ListWrapper
14 from tensorflow.python.trackable.data_structures import _DictWrapper
16 try:
```

After this following prompt, the answers were detailed, correct, and specific. Not much of a problem for Codex at all, though this assignment is certainly more terse than others.

CODEX

← Complete this assignment in its entirety please. Your code should be written in spots denoted by the word "Todo".

↑ Since your modifications would be overwritten.

```
5 """
----> 7 from keras import _tf_keras as _tf_keras
8 from keras import activations as activations
9 from keras import applications as applications
File ~/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages/keras/_tf_keras/init.py:1
----> 1 from keras._tf_keras import keras
File ~/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages/keras/_tf_keras/keras/init.py:7
1 """DO NOT EDIT.
2
...
----> 13 from tensorflow.python.trackable.data_structures import ListWrapper
14 from tensorflow.python.trackable.data_structures import _DictWrapper
16 try:
ModuleNotFoundError: No module named 'tensorflow'
```

Finished working

Updated the dependency cell at the top of `q_mup_coding.ipynb` to run `!pip install torch torchvision torchaudio keras matplotlib numpy tensorflow` so the `keras.datasets` import now brings along TensorFlow. After re-running that cell, the `ModuleNotFoundError: No module named 'tensorflow'` should be resolved.

Now that you have access to the generated figures as well, some of your answers may have been incorrect. Please use these figures and re-evaluate your answers to the questions. If you find no errors, leave as is and only change if necessary please.

Finished working

Updated all written responses to reflect the generated figures:

- **Part a answers (`q_mup_coding.ipynb` (line 5))** now cite the actual RMS values from the original and widened-layer plots (cell_4_1.png & cell_6_1.png), highlighting how RMS deltas scale with width before and after changing the architecture.
- **Part b answer (`q_mup_coding.ipynb` (line 7))** uses the three-panel norms plot (cell_8_0.png) to explain why Frobenius/Spectral norms best track activation changes, while the induced norm doesn't.
- **Part e answers (`q_mup_coding.ipynb` (line 13))** reference the learning-rate sweep figures for Adam and muP (cell_15_1.png & cell_16_1.png), describing how the optimal LRs shift with width in each case.
- **Shampoo discussion (`q_mup_coding.ipynb` (line 19))** now explicitly states the observed Frobenius vs. spectral relationship in the Adam and Shampoo plots and why orthogonalization makes the norms coincide.

No new code changes beyond these markdown updates.

Ask for follow-up changes

+ / Auto context

↑

Local Chat GPT-5.1-Codex Medium

45%

